

Natural Language-Based Command Option Retrieval Using a Local-First Approach: A Proof of Concept

Projektarbeit T3_3100

des Studiengangs **Informatik IT-Security** an der
Dualen Hochschule Baden-Württemberg Ravensburg
Campus Friedrichshafen

von **Leon Luca Eltrich**

Abgabedatum: **08.09.2025**

Bearbeitungszeitraum:	10 Wochen
Matrikelnummer, Kurs:	4276625, TIS23
Dualer Partner:	doubleSlash Net-Business GmbH, Friedrichshafen
Betreuer*in des Dualen Partners:	- - -
Gutachter*in der Dualen Hochschule:	Prof. Dr. Axel Hoff

Eigenständigkeitserklärung

Ich versichere hiermit, dass ich meine Projektarbeit mit dem Thema:

Natural Language-Based Command Option Retrieval Using a Local-First Approach: A Proof of Concept

selbstständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel benutzt habe. Ich versichere zudem, dass die eingereichte elektronische Fassung mit der gedruckten Fassung übereinstimmt.

Ort, Datum

Unterschrift

Abbreviations

LLM Large Language Model

Inhaltsverzeichnis

1	Introduction	1
1.1	Background and Motivation	1
1.2	The Problem	1
1.3	Proposed Solution: A Local-First Approach	2
1.4	Research Questions	2
1.5	Expected Contributions	2
2	Background	4
3	Methodology	5
4	Literature Review	6
5	Systems Design And Implementation	7
5.1	Design	7
5.1.1	Requirements	7
5.1.2	Business Cases?	10
5.2	Implementation	10
6	Evaluation	11
6.1	Technical Benchmarking	11
6.2	User Study	11
7	Results	12
8	Discussion	13
8.1	Validity Concerns	13
8.2	Answers To The Research Questions	13
9	Conclusion And Future Work	14

1 Introduction

1.1 Background and Motivation

For decades, the Command Line Interface (CLI) has served as a fundamental and powerful mechanism for navigating and managing computer systems. Command-line applications remain ubiquitous across software development, system administration, data analysis, and scientific computing. They offer unmatched efficiency, scriptability, and granular control over system resources.

However, despite its powerful features and long-standing existence, the command line continues to present a challenge, especially to novice users. The interface relies heavily on strict syntactical rules, cryptic abbreviations, and inconsistent command option flags across different utilities. While newcomers might struggle to interpret the highly technical descriptions found in traditional man pages or help manuals, the cognitive load affects seasoned users as well. Applications frequently possess an excessive number of options, flags, and arguments, making it difficult to utilize tools to their full potential without constant reference to external documentation.

1.2 The Problem

The traditional methods for bridging this knowledge gap, such as manual page consultation, web searches, or community forums, often require significant context switching, which disrupts the user's workflow. Recently, cloud-based Large Language Models (LLMs) such as ChatGPT or Gemini have emerged as popular alternatives for providing immediate command-line guidance.

While they offer highly intuitive natural language interaction, they suffer from several critical limitations in the context of system administration and software development:

- **Privacy and Security Risks:** Relying on cloud APIs requires transmitting user queries – which may contain sensitive system paths, file names, or proprietary tool names – to external servers.
- **Networking Constraints:** Cloud models are inherently unavailable in restricted, air-gapped, or offline environments, which are common in enterprise server management and secure development setups.
- **Domain-Specific Knowledge Gap:** General-purpose models often lack knowledge of internal, closed-source, or highly niche CLI applications specific to a particular organization's workflow.
- **Resource Usage:** While open-source LLMs exist, they tend to be quite hardware and resource heavy, which limits their feasibility.

1.3 Proposed Solution: A Local-First Approach

This thesis aims at addressing the challenges outlined in section 1.2 by exploring a local-first, privacy-preserving and extensible command option retrieval system. The core objective is to design, implement and evaluate a system that interprets natural-language prompts and accurately retrieves the correct command-line options without relying on external services.

By utilizing natural language processing approaches such as embeddings and lightweight language models, the proposed system will map user-generated descriptions of what they are trying to do (e.g. "List all files in this directory, including the ones not shown.") to the command-line options required for the task. Crucially, this proof-of-concept is designed to operate strictly off-line, with a focus on acceptable response times and hardware usage, utilizing only standard CPU hardware.

1.4 Research Questions

To systematically evaluate the feasibility and performance of the proposed local-first approach, this thesis seeks to address several core research questions. These questions are designed to bridge the gap between technical optimization and human-centric usability, focusing specifically on the constraints of offline, CPU-bound environments.

The study is guided by the following specific research questions:

- **RQ1:** Which local NLP techniques can effectively map natural language descriptions to command-line options with limited hardware resources available?
- **RQ2:** What trade-offs exist between accuracy, resource usage and extensibility in such a system?
- **RQ3:** How can the system be extended with new commands and options with minimal overhead?

1.5 Expected Contributions

This thesis provides both a practical tool and new scientific insights into how natural language can improve command-line interactions. The primary contributions include:

First, this work delivers a functional proof-of-concept system that proves it is possible to retrieve command options locally and privately. Second, it provides a detailed analysis of the technical trade-offs – balancing accuracy, speed, and memory usage – when running NLP models on standard CPU hardware.

Additionally, the thesis introduces an extensible architecture that allows developers to easily add natural language support to other CLI tools. Finally, the research offers an understanding

of the preceived effectiveness of the tool as experienced by users. Together, these results offer a practical path toward making the command line more accessible without compromising user privacy.

2 Background

3 Methodology

4 Literature Review

5 Systems Design And Implementation

5.1 Design

Database: - Tabelle für Tool beschreibungen - Tool name (ID) - Tool Beschreibung (TEXT) - Tool Beschreibung Vector (BLOB) - Tabelle für tool option Beschreibungen - option (ID) - Tool Name (ID) - Tool Option beschreibung (TEXT) - Tool Option Beschreibung Vector (BLOB)

Pipeline Design & Systemablaufdiagramm

2 systeme, eine lightweight api die das modell lädt und embeddings macht falls app registriert wird und das cli system

5.1.1 Requirements

Tabelle 1: Hardware and Environmental Constraints

ID	Constraint	Description
ENV-1	Target Hardware	All performance metrics must be achieved on a baseline Proxmox VM configured with 4 vCPUs (AMD Ryzen 7 5700X architecture), 8GB of 2133MHz DDR4 RAM, and a Samsung SSD 970 Evo Plus NVMe drive.
ENV-2	Offline Isolation	The system must function completely offline. It must not initiate any outbound network requests under any circumstances.
ENV-3	Zero Third-Party Daemon	The system must be entirely self-contained. While the system's own architecture may utilize background processes, it must not require the installation or execution of third-party standalone server software.

Tabelle 2: Functional Requirements (FR)

ID	Requirement	Description
FR-1	Command Generation	The system must accept natural language queries and return a syntactically correct command-line string comprising the base tool, relevant option flags, and placeholders for user-specific positional parameters.
FR-2	Non-Execution Boundary	The system must strictly output the generated command as text and never autonomously execute it in the host shell.
FR-3	Authenticated Ingestion	The system must provide a programmatic interface to ingest and index new tool catalogs, utilizing an authentication or cryptographic authorization mechanism to prevent unauthorized manipulation.
FR-4	Catalog Lifecycle Management	The system must provide authenticated interfaces allowing authorized entities to list, update, and remove previously ingested tool catalogs.
FR-5	State Persistence	The system must persist all ingested catalogs and computed retrieval states locally, ensuring immediate availability across system reboots and terminal sessions without requiring re-ingestion.
FR-6	Syntactical Validation	The system must logically evaluate generated commands to ensure no mutually exclusive flags are recommended simultaneously, and that any required positional ordering is strictly respected.
FR-7	Interactive Disambiguation	If a query is ambiguous, triggers a flag conflict, or falls outside the domain of ingested catalogs, the system must halt and prompt the user for clarification rather than returning an invalid command.

Tabelle 3: Non-Functional Requirements (NFR)

ID	Requirement	Description
NFR-1	Query Latency	The system must return a completed command recommendation in under 1000ms from the moment the query is submitted.
NFR-2	Cold Startup Latency	The application must initialize and be ready to process input in under 1000ms.
NFR-3	Memory Bounding	The peak RAM utilization of the system process must never exceed 1024 MB at any time during operation.
NFR-4	Latency Scalability	The system must maintain its <1000ms query latency when the persistent state is scaled to hold definitions for up to Average number of CLI tools installed on a linux machine distinct tools.
NFR-5	Retrieval Quality	The system must achieve a minimum Macro-Averaged F1-Score of [Target] and a Jaccard Similarity of [Target] against the established evaluation dataset.
NFR-6	Algorithmic Modularity	The system architecture must decouple the natural language processing and search logic from the user interfaces, allowing retrieval models to be swapped without altering the broader application.
NFR-7	Zero-Runtime Portability	The system must be deployable as self-contained artifacts for Linux, macOS, and Windows, requiring no pre-installed language runtimes on the host machine.
NFR-8	IPC Network Isolation	Any Inter-Process Communication (IPC) mechanisms utilized by the system must be strictly bound to local OS constructs, explicitly denying any non-local network access.
NFR-9	State Access Control	The system must utilize host OS access control mechanisms to ensure its persistent state data is logically isolated and protected from unauthorized modification by other non-privileged users.
NFR-10	Concurrency Resilience	The system must safely process simultaneous execution requests from multiple terminal instances without causing deadlocks, process crashes, or state corruption.
NFR-11	Input Sanitization	The system must sanitize all user inputs and programmatic payloads to prevent injection attacks against the underlying persistent state mechanism.
NFR-12	Opt-In Diagnostic Logging	By default, any diagnostic logs generated by the system must be strictly scrubbed of the user's raw natural language queries. Unredacted query logging must require explicit user opt-in.

5.1.2 Business Cases?

user erfragt tool das kein options erfordert user erfragt tool das eine option erfordert user erfragt tool das mehrere options erfordert

entwickler möchte das modell austauschen entwickler möchten ihre anwendung registrieren

5.2 Implementation

- Rust

Beschreibung der Module und wie sie ineinandergreifen (Diagram?)

tar, mv, cp,

6 Evaluation

6.1 Technical Benchmarking

6.2 User Study

7 Results

8 Discussion

8.1 Validity Concerns

8.2 Answers To The Research Questions

9 Conclusion And Future Work

Abbildungsverzeichnis

Tabellenverzeichnis

1	Hardware and Environmental Constraints	7
2	Functional Requirements (FR)	8
3	Non-Functional Requirements (NFR)	9
4	Eingesetzte KI-Werkzeuge und Beschreibungen deren Verwendung	17

KI-Verzeichnis

Werkzeug	Beschreibung der Nutzung
ChatGPT	Der Einsatz erfolgte in der gesamten Arbeit zur sprachlichen Gestaltung. Konkret diente es dazu, vorhandene Texte stilistisch an den wissenschaftlichen Schreibstil anzupassen und präzise Formulierungen zu entwickeln. Außerdem wurde wie in Kapitel ?? bzw. Kapitel ?? beschrieben, die Funktion <i>ChatGPT Deep Research</i> für das Identifizieren relevanter Quellen genutzt. Limitationen wurden in besagten Kapiteln reflektiert. Darüber hinaus kam ChatGPT zur Klärung fachlicher Verständnisfragen im Verlauf der Recherche und Umsetzung des Projektes zum Einsatz.
Perplexity	Der Einsatz erfolgte wie in den Kapiteln ?? und ?? beschrieben, um relevante Quellen zu identifizieren. Wie bei <i>ChatGPT Deep Research</i> , wurde der Einsatz in den Kapiteln kritisch hinterfragt und Limitationen identifiziert.

Tabelle 4: Eingesetzte KI-Werkzeuge und Beschreibungen deren Verwendung